About Hire Hub Project

🌐 HireHub — The Complete Project Brain Dump

What Exactly Are We Building?

Think of it as a **simplified LinkedIn Jobs / Naukri.com** — but one YOU built from scratch.

```
HireHub is a platform where:

EMPLOYERS can:                SEEKERS can:
──────────────                ────────────

✅ Register a company         ✅ Browse all jobs
✅ Post job listings          ✅ Search & filter jobs
✅ See who applied            ✅ Apply with resume
✅ Accept / Reject            ✅ Track application status
✅ Manage their posts         ✅ Manage their profile
```

🗺️ The Big Picture — System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        BROWSER                               │
│                                                              │
│    ┌──────────────────────────────────────────────────┐      │
│    │              REACT FRONTEND                      │      │
│    │            (localhost:3000)                      │      │
│    │                                                  │      │
│    │  Pages:                                          │      │
│    │  / ──────── Home (job listings)                  │      │
│    │  /login ─── Login page                           │      │
│    │  /register── Register page                       │      │
│    │  /jobs ───── Browse jobs                         │      │
│    │  /jobs/:id── Job detail                          │      │
│    │  /dashboard─ Employer dashboard                  │      │
│    │  /profile ── Seeker profile                      │      │
│    └──────────────────────┬───────────────────────────┘      │
│                           │                                  │
│                 Axios HTTP Calls (JSON)                      │
│                 Authorization: Bearer <JWT>                  │
│                           │                                  │
└───────────────────────────┼──────────────────────────────────┘
                            │
                            ▼

┌──────────────────────────────────────────────────────────────┐
```

```
│              DJANGO + DRF BACKEND                          │
│                (localhost:8000)                            │
│                                                            │
│   API Endpoints:                                           │
│   POST /api/auth/register/                                 │
│   POST /api/auth/login/                                    │
│   GET  /api/jobs/          (public)                        │
│   POST /api/jobs/          (employer only)                 │
│   GET  /api/jobs/:id/      (public)                        │
│   POST /api/jobs/:id/apply (seeker only)                   │
│   GET  /api/applications/  (my applications)               │
│   PATCH/api/applications/:id/ (employer updates status)    │
│                                                            │
└────────────────────────────────────────────────────────────┘
                          │
                          │
                          ▼

┌────────────────────────────────────────────────────────────┐
│                  POSTGRESQL DATABASE                       │
│                                                            │
│   Tables:                                                  │
│   users, companies, job_posts, applications                │
│                                                            │
└────────────────────────────────────────────────────────────┘
                          │
                          ▼

┌────────────────────────────────────────────────────────────┐
│              FILE STORAGE (local / S3 later)               │
│   Stores: resumes (PDF), avatars, company logos            │
└────────────────────────────────────────────────────────────┘
```

🔄 Complete User Flow — Every Journey

Journey 1: Employer Posts a Job

```
1. Alice opens HireHub in browser (React loads)
        │
2. Alice clicks "Register as Employer"
   React sends → POST /api/auth/register/
                  { username, email, password, role: "EMPLOYER" }
        │
3. Django creates User(role=EMPLOYER) in DB
   Django responds → { access_token, refresh_token, user_data }
   React stores token in memory (or localStorage)
        │
4. Alice fills "Create Company" form
   React sends → POST /api/company/
```

```
                    Authorization: Bearer <token>
                    { name: "TechCorp", location: "Pune" }
         |
5. Django checks: is this user an EMPLOYER? ✅
   Creates Company linked to Alice in DB
         |
6. Alice clicks "Post a Job"
   React sends → POST /api/jobs/
                    Authorization: Bearer <token>
                    { title, description, skills, salary... }
         |
7. Django creates JobPost linked to Alice + TechCorp
   Responds → { id: 1, title: "Python Dev", ... }
         |
8. Job is now LIVE and visible to all seekers! 🎉
```

Journey 2: Seeker Finds & Applies to a Job

```
1. Bob opens HireHub (doesn't need to login to browse!)
         |
2. Bob sees job listings
   React sends → GET /api/jobs/
   Django responds → list of all active jobs (JSON)
   React renders them as cards on screen
         |
3. Bob types "Python" in search box
   React sends → GET /api/jobs/?search=Python&location=Pune
   Django filters DB, responds with matching jobs
         |
4. Bob clicks on "Python Dev at TechCorp"
   React sends → GET /api/jobs/1/
   Django responds → full job detail JSON
   React renders full job page
         |
5. Bob clicks "Apply Now"
   (React checks: is Bob logged in? No → redirect to login)
         |
6. Bob registers as SEEKER, gets JWT token
         |
7. Bob fills application form, uploads resume PDF
   React sends → POST /api/jobs/1/apply/
                    Authorization: Bearer <token>
                    Form data: { cover_letter, resume: <file> }
         |
8. Django checks:
   - Is user logged in? ✅ (JWT valid)
```

```
   - Is user a SEEKER? ✅
   - Has user already applied? ❌ (unique_together check)
   Creates Application(status=PENDING) in DB
   Saves resume PDF to /media/resumes/
        |
9. Bob sees "Application Submitted! 🎉"
   Bob can now track status on his dashboard
```

Journey 3: Employer Reviews Applications

```
1. Alice logs in, goes to her dashboard
   React sends → GET /api/jobs/1/applications/
                 Authorization: Bearer <token>
        |
2. Django checks: does this job belong to Alice? ✅
   Returns list of all applications with applicant info
        |
3. Alice sees Bob's application
   She downloads his resume (file from /media/resumes/)
   She decides to accept him!
        |
4. Alice clicks "Accept"
   React sends → PATCH /api/applications/1/
                 Authorization: Bearer <token>
                 { status: "ACCEPTED" }
        |
5. Django updates Application.status = "ACCEPTED" in DB
        |
6. Bob refreshes his dashboard
   React sends → GET /api/applications/mine/
   Django returns Bob's applications with updated status
   Bob sees: "TechCorp - Python Dev → ACCEPTED! 🎉"
```

🧠 Full Stack Developer Mental Model

This is how you should THINK when building any full stack project:

```
┌──────────────────────────────────────────────┐
│  QUESTION 1: "What data do I need to store?"  │
│  → Think in MODELS (database design first)    │
│  → Draw the relationships                     │
│  → User, Company, JobPost, Application        │
└──────────────────────────────────────────────┘
```

```
                        ↓
┌──────────────────────────────────────────────────┐
│  QUESTION 2: "What actions can users take?"      │
│  → Think in API ENDPOINTS                        │
│  → Each action = one endpoint                    │
│  → Register, Login, Post Job, Apply, etc.        │
└──────────────────────────────────────────────────┘
                        ↓
┌──────────────────────────────────────────────────┐
│  QUESTION 3: "Who is allowed to do what?"        │
│  → Think in PERMISSIONS                          │
│  → Public: browse jobs                           │
│  → Employer only: post jobs, see applications    │
│  → Seeker only: apply to jobs                    │
│  → Owner only: edit/delete their own content     │
└──────────────────────────────────────────────────┘
                        ↓
┌──────────────────────────────────────────────────┐
│  QUESTION 4: "What does the user SEE?"           │
│  → Think in PAGES / COMPONENTS (React)           │
│  → Each page consumes which API?                 │
│  → What state do I need to manage?               │
└──────────────────────────────────────────────────┘
                        ↓
┌──────────────────────────────────────────────────┐
│  QUESTION 5: "How do I deploy this?"             │
│  → Think in ENVIRONMENTS                         │
│  → Dev (local) vs Production (live server)       │
│  → Env variables, static files, DB config        │
└──────────────────────────────────────────────────┘
```

📋 Every API Endpoint We'll Build

```
AUTH
────

POST   /api/auth/register/        → Create account (Employer/Seeker)
POST   /api/auth/login/           → Get JWT tokens
POST   /api/auth/token/refresh/   → Refresh access token
GET    /api/auth/me/              → Get my profile
PATCH  /api/auth/me/              → Update my profile

COMPANY
───────

POST   /api/company/              → Create company (Employer only)
GET    /api/company/:id/          → View a company
PATCH  /api/company/:id/          → Update company (owner only)
```

```
JOBS
────
GET    /api/jobs/                 → List all active jobs (public)
POST   /api/jobs/                 → Create a job (Employer only)
GET    /api/jobs/:id/             → Job detail (public)
PATCH  /api/jobs/:id/             → Edit job (owner only)
DELETE /api/jobs/:id/             → Delete job (owner only)
GET    /api/jobs/mine/            → My posted jobs (Employer only)

APPLICATIONS
────────────
POST   /api/jobs/:id/apply/       → Apply to a job (Seeker only)
GET    /api/applications/mine/    → My applications (Seeker)
GET    /api/jobs/:id/applications/→ Job's applications (Employer)
PATCH  /api/applications/:id/     → Update status (Employer only)
```

🖥️ Every React Page We'll Build

```
PUBLIC PAGES (no login needed)
──────────────────────────────
/                   → Home — featured jobs, hero section
/jobs               → Browse all jobs with search + filters
/jobs/:id           → Single job detail page
/register           → Register form (choose role!)
/login              → Login form

SEEKER PAGES (login required)
─────────────────────────────
/dashboard          → My applications + status tracker
/profile            → Edit my profile, upload avatar

EMPLOYER PAGES (login + EMPLOYER role required)
───────────────────────────────────────────────
/employer/dashboard     → My posted jobs overview
/employer/jobs/new      → Create new job form
/employer/jobs/:id/edit → Edit existing job
/employer/jobs/:id/applications → See who applied
```

🔐 Permission Matrix — Who Can Do What

```
                    Guest  Seeker  Employer  Admin
                    ─────  ──────  ────────  ─────
```

```
Browse jobs          ✅      ✅      ✅      ✅
View job detail      ✅      ✅      ✅      ✅
Register/Login       ✅      ✅      ✅      ✅
Apply to job         ❌      ✅      ❌      ✅
View my applications ❌      ✅      ❌      ✅
Post a job           ❌      ❌      ✅      ✅
Edit own job         ❌      ❌      ✅      ✅
Delete own job       ❌      ❌      ✅      ✅
View applicants      ❌      ❌      ✅      ✅
Update app status    ❌      ❌      ✅      ✅
Access admin panel   ❌      ❌      ❌      ✅
```

This matrix is what we'll translate directly into **DRF Permissions** in code.

🗂️ How Frontend & Backend Talk — The Token Flow

```
1. LOGIN
   ─────
   React → POST /api/auth/login/ { email, password }
   Django → { access: "eyJ...", refresh: "eyJ..." }
   React stores access token in memory

2. EVERY PROTECTED REQUEST
   ──────────────────────
   React → GET /api/jobs/mine/
           Header: Authorization: Bearer eyJ...
   Django → reads header → decodes JWT → finds user → checks permission
   Django → returns data ✅ or 401 Unauthorized ❌

3. TOKEN EXPIRES (after 60 min)
   ──────────────────────────
   React → POST /api/auth/token/refresh/ { refresh: "eyJ..." }
   Django → { access: "eyJ...NEW TOKEN..." }
   React updates stored token, retries original request
```

🏗️ Build Order — Why We Build in This Sequence

```
Step 1: Models (Database) ← FOUNDATION. Everything depends on this.
         ↓
Step 2: DRF Serializers ← Convert DB objects ⇔ JSON
         ↓
Step 3: DRF Views/APIs ← The actual endpoints
         ↓
```

```
Step 4: Permissions ← Lock down who can do what
         ↓
Step 5: Test APIs (Postman) ← Make sure backend works BEFORE frontend
         ↓
Step 6: React Frontend ← Now consume the working API
         ↓
Step 7: Connect + Polish ← Wire everything together
         ↓
Step 8: Deploy ← Push live, get real URL
```

Why this order?

```
If you build React first:
  "Where's my data coming from?" → You have to mock it
  "What shape is the JSON?" → You don't know yet
  → Lots of rework! ❌

If you build backend first:
  Test every endpoint in Postman ✅
  Know EXACTLY what JSON the frontend receives ✅
  React just consumes — no guessing! ✅
```

🎯 What This Project Proves to an Interviewer

When you show HireHub in an interview, here's what they see:

```
"You built a custom user model"
→ You understand Django auth system deeply

"You have two user roles with different permissions"
→ You understand authorization, not just authentication

"You built REST APIs with DRF"
→ You can build production-grade backends

"Your React frontend consumes a real API"
→ You understand full stack data flow

"You used JWT authentication"
→ You know modern, scalable auth patterns

"It's deployed with a live URL"
→ You know DevOps basics, not just local development

"You have file uploads (resumes)"
→ You've handled real-world complexity
```

```
"You used PostgreSQL with proper relationships"
→ You understand relational database design
```

One project. Eight skills demonstrated. That's why this project wins. 🏆